PRODUCT REQUIREMENTS DOCUMENT

Full-Stack Ticket Triage & Escalation System

NimbusTech Solutions — Phase 2: The Full-Stack Build

Document Type: Capstone Project Brief (PRD)
Client: NimbusTech Solutions
Prepared For: Intern Engineering Batch
Version: 1.0
Status: Approved for Build

Document Control

Project Name	Full-Stack Ticket Triage & Escalation System
Client	NimbusTech Solutions (B2B SaaS, project-management software)
Requested By	Engineering Manager, Support Tooling Team
Audience	Intern engineers building the MVP end to end
Prerequisite Skills	Python scripting best practices, PEP8, basic FastAPI, basic HTML/CSS/ JavaScript (fetch API)
Scope Level	MVP — deliberately simple stack, no database, no auth, no frameworks

Table of Contents

1. Executive Summary **3**

2. Business Context & Client Scenario **3**

3. Objectives & Success Metrics **3**

4. Scope **4**

5. System Architecture **4**

6. Functional Requirements — Backend **5**

7. Functional Requirements — Frontend **6**

8. API Contract **7**

9. Non-Functional Requirements **8**

10. Suggested Project Structure **9**

11. UI Requirements & Wireframe **10**

12. Suggested Build Order (Milestones) **11**

13. Acceptance Criteria (Definition of Done) **11**

14. Evaluation Rubric **12**

15. Stretch Goals **12**

Appendix A — Sample Data **13**

Appendix B — Expected Report Shape **13**

1. Executive Summary

NimbusTech's support-ops team previously commissioned a small MVP (Phase 1) consisting of a data-processing script and a basic backend API for triaging support tickets. That phase proved the concept internally, but the API is currently only usable via `/docs` or command-line tools — not something a non-technical support-ops analyst can use day to day.

Phase 2 asks your team to complete the picture: combine everything learned so far — **Python scripting best practices**, **PEP8**, a **basic FastAPI backend**, and **basic HTML/CSS/JavaScript** — into one working, full-stack MVP: a real backend and a real browser-based dashboard that talk to each other.

2. Business Context & Client Scenario

NimbusTech Solutions is a B2B SaaS company providing project-management software to roughly 200 corporate clients. Every night, their legacy helpdesk tool exports all support tickets as a CSV file. An analyst used to review this manually — a slow, error-prone process that occasionally missed SLA breaches until a client complained.

Phase 1 replaced the manual review with an automated script (which validates and classifies tickets) and a basic API (which exposes that data over HTTP). **Phase 2's ask**, directly from the support-ops team lead:

> **Client quote (paraphrased ask):** "The API is great for developers, but my analysts aren't going to open a terminal or a Swagger page every morning. We need an actual screen — something you click through — that shows today's tickets, highlights the ones breaching SLA, and lets us update or close a ticket without calling engineering every time."

3. Objectives & Success Metrics

- **Objective 1:** Support-ops staff can view all tickets and immediately see which ones have breached SLA, without any technical tooling.
- **Objective 2:** Support-ops staff can create, update, and close tickets directly from the browser.
- **Objective 3:** The whole system stays simple and maintainable — no database, no authentication, no frontend framework — this is an MVP, not the final product.

Success looks like:

- An analyst can open one HTML page and immediately answer "which tickets need attention right now?"
- No manual CSV review is needed for day-to-day triage anymore.
- The codebase is clean enough that a future team can safely extend it (add a database, add auth) without a rewrite.

4. Scope

In Scope

- A ticket-processing script (reused/refined from Phase 1)
- A basic FastAPI backend exposing ticket data over HTTP (CRUD + a breached-tickets view)
- A plain HTML/CSS/JavaScript frontend that consumes that API and is usable by a non-technical analyst

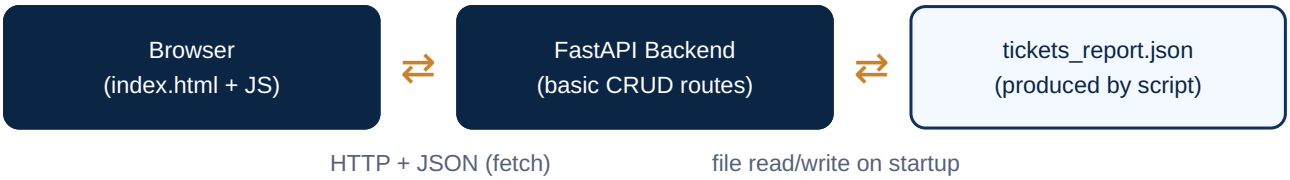
Out of Scope (explicitly, for this phase)

- Any real database (PostgreSQL, MySQL, etc.) — in-memory / JSON-file storage is sufficient
- Authentication or user accounts/login
- Any frontend framework (React, Vue, Angular) or build tooling (Webpack, Vite, npm)
- Advanced FastAPI features — middleware chains, dependency injection, background tasks, WebSockets
- Mobile app version, email notifications, or Slack integration

> **Important:** "Basic" is a requirement, not a limitation of your skill. The client explicitly wants a lean, simple MVP right now — solutions that are more complex than required will be marked down, not up.

5. System Architecture

The system has three moving pieces, each mapping directly to a skill you've already practiced:



The **script** (Module 1) reads the raw CSV, validates and classifies tickets, and writes a clean JSON report. The **backend** (Module 2) loads that report at startup and serves it over a handful of simple REST routes, with CORS enabled so a separately-opened HTML file can call it. The **frontend** (Module 3) is a static page that calls those routes with `fetch()` and renders the results — no logic is duplicated on the frontend; it only displays what the backend gives it.

6. Functional Requirements — Backend

6.1 Processing Script (Module 1)

ID	Requirement
FR-B1	Read the raw tickets CSV: `ticket_id, customer_name, category, priority_raw, created_at, sla_hours, status`
FR-B2	Validate every row; a row is invalid if `ticket_id` is missing, `sla_hours` is not a positive number, `created_at` doesn't parse, or `priority_raw` isn't one of `low/medium/high/critical`
FR-B3	Invalid rows are collected separately, never silently dropped and never crash the script
FR-B4	Compute SLA breach per ticket: breached if `created_at + sla_hours` is before "now" and `status != closed`
FR-B5	Assign a numeric priority score (`low=1, medium=2, high=3, critical=4`)
FR-B6	Compute summary counts: total, valid, invalid, breached, and a breakdown by category
FR-B7	Abort entirely (no report written, non-zero exit code) if the invalid-row ratio exceeds 10%
FR-B8	Write the structured JSON report described in Appendix B
FR-B9	Runnable from the CLI with `argparse`, e.g. `python main.py --input data/tickets_raw.csv --output data/tickets_report.json`

6.2 API Backend (Module 2)

ID	Requirement
FR-B10	Load `tickets_report.json` into memory on application startup
FR-B11	`GET /` returns a basic status/welcome payload
FR-B12	`GET /tickets` returns the full in-memory ticket list
FR-B13	`GET /tickets/breached` returns only SLA-breached tickets
FR-B14	`GET /tickets/summary` returns the report's summary block (new in Phase 2 — the dashboard needs this)
FR-B15	`GET /tickets/{id}` returns one ticket, or `404` if it doesn't exist
FR-B16	`POST /tickets` creates a new ticket from a JSON body
FR-B17	`PUT /tickets/{id}` or `PATCH /tickets/{id}` updates `status`/`priority_raw`
FR-B18	`DELETE /tickets/{id}` removes a ticket
FR-B19	CORS must be enabled so a separately-opened frontend file can call every route above

7. Functional Requirements — Frontend

The frontend is **plain HTML/CSS/JavaScript** — one or more static files, no framework, no build step.

ID	Requirement
FR-F1	On page load, call `GET /tickets` and render every ticket as a row in a table
FR-F2	A summary panel at the top shows total / valid / breached counts, from `GET /tickets/summary`
FR-F3	A "Show only breached" toggle switches the table between `GET /tickets` and `GET /tickets/breached`
FR-F4	Breached tickets are visually highlighted in the table (e.g. red background/badge)
FR-F5	A simple form creates a new ticket via `POST /tickets`; on success, the new row appears in the table without a full page reload
FR-F6	Each row has a status control (e.g. a dropdown) that calls `PUT/PATCH /tickets/{id}` when changed
FR-F7	Each row has a "Delete" button that calls `DELETE /tickets/{id}` and removes the row on success
FR-F8	If the API call fails (network error, non-2xx response), show a clear, human-readable error message in the UI — never a silent failure or a raw JS error in the console only
FR-F9	If there are zero tickets, show a friendly empty-state message instead of a blank table

> **Reminder:** the frontend should contain **zero business logic** — no computing SLA breaches, no priority scoring in JavaScript. All of that already happened in the backend/report. The frontend's job is strictly to call the API and render what it gets back.

8. API Contract

Method	Path	Description	Success	Errors
GET	/	Health/status check	200	—
GET	/tickets	List all tickets	200	—
GET	/tickets/breached	List only SLA-breached tickets	200	—
GET	/tickets/summary	Report summary counts	200	—
GET	/tickets/{id}	Get one ticket by id	200	404
POST	/tickets	Create a new ticket (JSON body)	201	422
PUT/ PATCH	/tickets/{id}	Update status / priority_raw	200	404, 422
DELETE	/tickets/{id}	Delete a ticket	200	404

Example — POST /tickets request body

```
{
  // all fields required for creation
  "customer_name": "Wayne Enterprises",
  "category": "technical",
  "priority_raw": "high",
  "sla_hours": 24,
  "status": "open"
}
```

9. Non-Functional Requirements

ID	Requirement
NFR1	All Python code follows PEP8 — naming, spacing, import grouping, line length
NFR2	Script structure follows professional convention: constants, SRP functions, `main()` + `__name__` guard
NFR3	No hardcoding — thresholds, mappings, and paths are constants/config, not magic values
NFR4	DRY — no duplicated logic across script, API, or frontend
NFR5	`logging` used in Python code instead of `print()`
NFR6	Errors handled gracefully everywhere — backend never returns a raw traceback; frontend never fails silently
NFR7	Every Python function has a docstring
NFR8	A virtual environment + `requirements.txt` is used for the backend
NFR9	The FastAPI backend stays basic — no database, no auth, no middleware beyond CORS, no dependency injection chains
NFR10	The frontend uses plain HTML/CSS/JavaScript only — no framework, no build tools
NFR11	Frontend and backend are cleanly separated — the frontend only communicates via `fetch()` over HTTP, never by importing backend code
NFR12	Project is organized across multiple files by responsibility — no single giant file for the whole system

10. Suggested Project Structure

```
nimbustech_fullstack/
## backend ##
├── venv/                        # not committed
├── requirements.txt
├── README.md
├── data/
│   ├── tickets_raw.csv
│   └── tickets_report.json      # generated by the script
├── ticket_processor/
│   ├── __init__.py
│   ├── config.py                # constants: thresholds, priority map
│   ├── models.py                # Ticket data structure
│   ├── validators.py            # row validation logic
│   ├── processor.py             # classification + report building
│   └── main.py                  # argparse + main() + logging
└── ticket_api/
    ├── __init__.py
    ├── main.py                  # FastAPI app, routes, CORS setup
    ├── models.py                # Pydantic request/response models
    └── store.py                 # loads report, in-memory ticket list

## frontend (new in Phase 2) ##
frontend/
├── index.html                   # page structure, table, forms
├── styles.css                   # all styling, kept separate from HTML
└── app.js                       # all fetch() calls + DOM rendering logic
```

> **Note:** keep `styles.css` and `app.js` in separate files from `index.html` — this is the same Single Responsibility Principle you already apply in Python, just applied to a frontend project.

11. UI Requirements & Wireframe

A rough sketch of the expected dashboard layout — exact visual design is up to you, but the layout and behavior below should be present:

- Top bar: dashboard title + live summary counts
- Controls row: breached-only filter toggle, "New Ticket" button that reveals a small form
- Table rows: id, customer, priority, an editable status control, and a delete action
- Breached rows are visually distinct (color/badge) from normal rows — see wireframe above

12. Suggested Build Order (Milestones)

1. **Milestone 1 — Backend script.** Reuse/refine Module 1 from Phase 1. Confirm the report JSON matches Appendix B.
2. **Milestone 2 — Backend API.** Build all FR-B10 to FR-B19 routes, including the new `/tickets/summary` route and CORS. Verify everything manually via `/docs` first.
3. **Milestone 3 — Static frontend shell.** Build `index.html` + `styles.css` with the table/summary panel layout (static/fake data first, no fetch yet).
4. **Milestone 4 — Wire up the frontend.** Replace the fake data with real `fetch()` calls for load, filter, create, update, delete.
5. **Milestone 5 — Polish.** Error states, empty states, PEP8 clean-up on the Python side, and a README explaining how to run all three pieces together.

13. Acceptance Criteria (Definition of Done)

- ☐ Backend script produces a correct report and aborts cleanly on >10% invalid rows
- ☐ `uvicorn ticket_api.main:app --reload` starts the API and all 8 routes work via `/docs`
- ☐ Opening `frontend/index.html` directly in a browser (with the API running) loads and displays real ticket data — no CORS errors in the console
- ☐ Creating, updating, and deleting a ticket from the UI is reflected immediately without a full page reload
- ☐ Breached tickets are visually distinguishable in the table
- ☐ An API failure (e.g. server not running) shows a readable message in the UI, not a blank page or console-only error
- ☐ All Python code is PEP8-clean, documented, and free of hardcoded values
- ☐ Project structure matches Section 10 — frontend and backend cleanly separated

14. Evaluation Rubric

Area	What's checked
Correctness	Script classifies/validates correctly; all API routes and all UI interactions behave correctly end to end
Scripting & PEP8	SRP, constants, DRY, docstrings, no hardcoding, formatting
API design	Sensible REST routes, correct status codes, working CORS
Frontend quality	Clean separation of HTML/CSS/JS, no duplicated business logic, graceful error/empty states
Integration	Frontend and backend genuinely work together live, not just individually
Simplicity	Solution stays within the "basic" scope requested — no unnecessary complexity added

15. Stretch Goals (optional)

- A search box that filters the table by customer name or ticket id, client-side
- Sort the table by priority score, descending
- Auto-refresh the dashboard every 30 seconds using `setInterval`
- A simple category filter dropdown backed by `GET /tickets?category=...`

Only attempt these after every core requirement above is fully working.

Appendix A — Sample Input Data

```
ticket_id,customer_name,category,priority_raw,created_at,sla_hours,status
T-1001,Acme Corp,billing,high,2026-07-20 09:00:00,24,open
T-1002,Globex Inc,technical,critical,2026-07-19 14:30:00,8,open
T-1003,Initech,technical,medium,2026-07-21 10:00:00,48,closed
T-1004,,billing,low,2026-07-20 11:00:00,72,open
T-1005,Umbrella LLC,technical,critical,2026-07-18 08:00:00,-5,open
T-1006,Hooli,onboarding,medium,not-a-date,24,open
T-1007,Stark Industries,billing,unknown,2026-07-21 09:00:00,24,open
T-1008,Wayne Enterprises,technical,high,2026-07-19 09:00:00,24,open
```

Appendix B — Expected Report Shape

```
{
  "generated_at": "2026-07-23T09:15:00",
  "summary": {
    "total_rows": 8,
    "valid_tickets": 4,
    "invalid_rows": 4,
    "breached_count": 2,
    "by_category": { "billing": 1, "technical": 3 }
  },
  "tickets": [
    {
      "ticket_id": "T-1001",
      "customer_name": "Acme Corp",
      "category": "billing",
      "priority_raw": "high",
      "priority_score": 3,
      "created_at": "2026-07-20T09:00:00",
      "sla_hours": 24,
      "status": "open",
      "sla_breached": true
    }
  ],
  "invalid_rows": [
    { "raw_row": "T-1004,,billing,...", "reason": "missing customer_name" }
  ]
}
```